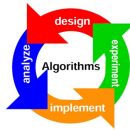


Estrategias algorítmicas

José A. Rodríguez Melquiades

29 de junio de 2026

Programación dinámica



- 1 Programación dinámica
- 2 Aplicaciones
- 3 Problema de la mochila binaria

Programación Dinámica - PD

- **PD:** Estrategia empleada para casos, generalmente, expresados en forma recursiva, donde un problema se divide en subproblemas los cuales a su vez son resueltos de manera recursiva cuyas soluciones se almacenan en una tabla para evitar hacer cálculos redundantes.

Programación Dinámica - PD

- **PD:** Estrategia empleada para casos, generalmente, expresados en forma recursiva, donde un problema se divide en subproblemas los cuales a su vez son resueltos de manera recursiva cuyas soluciones se almacenan en una tabla para evitar hacer cálculos redundantes.
- **Características:**

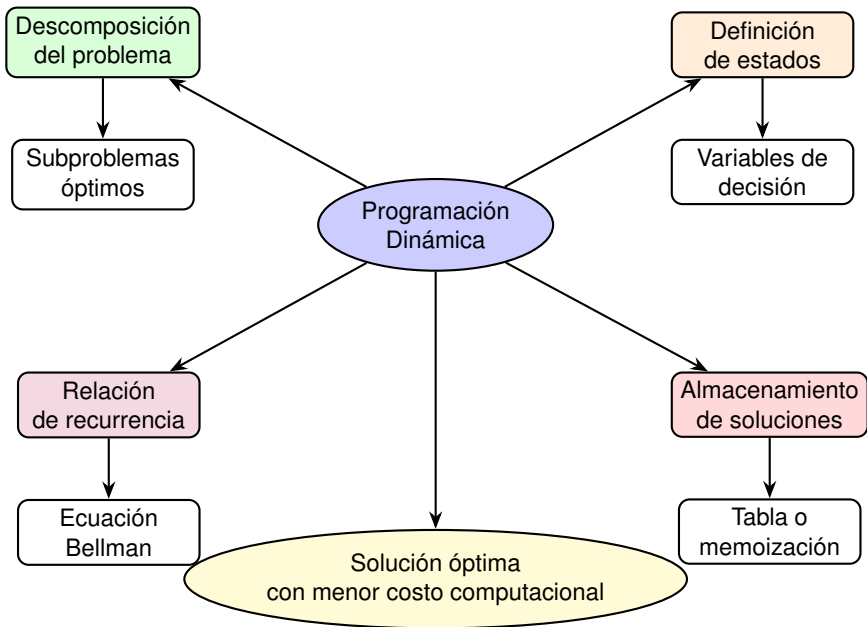
- **PD:** Estrategia empleada para casos, generalmente, expresados en forma recursiva, donde un problema se divide en subproblemas los cuales a su vez son resueltos de manera recursiva cuyas soluciones se almacenan en una tabla para evitar hacer cálculos redundantes.
- **Características:**
 - ▶ **Descomposición:** Divide un problema complejo en subproblemas.

- **PD:** Estrategia empleada para casos, generalmente, expresados en forma recursiva, donde un problema se divide en subproblemas los cuales a su vez son resueltos de manera recursiva cuyas soluciones se almacenan en una tabla para evitar hacer cálculos redundantes.
- **Características:**
 - ▶ **Descomposición:** Divide un problema complejo en subproblemas.
 - ▶ **Estados:** Representan la información para tomar decisiones.

- **PD:** Estrategia empleada para casos, generalmente, expresados en forma recursiva, donde un problema se divide en subproblemas los cuales a su vez son resueltos de manera recursiva cuyas soluciones se almacenan en una tabla para evitar hacer cálculos redundantes.
- **Características:**
 - ▶ **Descomposición:** Divide un problema complejo en subproblemas.
 - ▶ **Estados:** Representan la información para tomar decisiones.
 - ▶ **Recurrencia:** Relación óptima entre etapas (principio de optimalidad de Bellman).

- **PD:** Estrategia empleada para casos, generalmente, expresados en forma recursiva, donde un problema se divide en subproblemas los cuales a su vez son resueltos de manera recursiva cuyas soluciones se almacenan en una tabla para evitar hacer cálculos redundantes.
- **Características:**
 - ▶ **Descomposición:** Divide un problema complejo en subproblemas.
 - ▶ **Estados:** Representan la información para tomar decisiones.
 - ▶ **Recurrencia:** Relación óptima entre etapas (principio de optimalidad de Bellman).
 - ▶ **Memoria:** Evita cálculo redundantes mediante tablas o memoización.

- **PD:** Estrategia empleada para casos, generalmente, expresados en forma recursiva, donde un problema se divide en subproblemas los cuales a su vez son resueltos de manera recursiva cuyas soluciones se almacenan en una tabla para evitar hacer cálculos redundantes.
- **Características:**
 - ▶ **Descomposición:** Divide un problema complejo en subproblemas.
 - ▶ **Estados:** Representan la información para tomar decisiones.
 - ▶ **Recurrencia:** Relación óptima entre etapas (principio de optimalidad de Bellman).
 - ▶ **Memoria:** Evita cálculo redundantes mediante tablas o memoización.
 - ▶ **Resultado:** Obtiene una solución óptima con menor esfuerzo computacional.



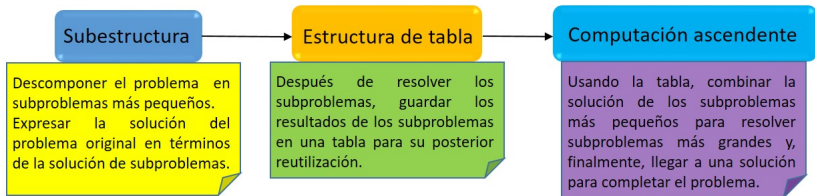


Figura 1: Estructura de programación dinámica.

- ¿ Cuando usar la PD ?:

- ¿ Cuando usar la PD ?:
 - ▶ Problema debe tener: Una formulación recursiva.

- ¿ Cuando usar la PD ?:

- ▶ Problema debe tener: Una formulación recursiva.
- ▶ Número total de instancias: Debe ser pequeño (n).

- ¿ Cuando usar la PD ?:

- ▶ Problema debe tener: Una formulación recursiva.
- ▶ Número total de instancias: Debe ser pequeño (n).
- ▶ PD presenta subestructura óptima: Solución óptima para el problema esta constituido de soluciones óptimas de los subproblemas.

- ¿ Cuando usar la PD ?:
 - ▶ Problema debe tener: Una formulación recursiva.
 - ▶ Número total de instancias: Debe ser pequeño (n).
 - ▶ PD presenta subestructura óptima: Solución óptima para el problema esta constituido de soluciones óptimas de los subproblemas.
- Ventaja: Una vez solucionado un subproblema, la respuesta se guarda en una tabla y nunca mas se debe recalcular.

- ¿ Cuando usar la PD ?:
 - ▶ Problema debe tener: Una formulación recursiva.
 - ▶ Número total de instancias: Debe ser pequeño (n).
 - ▶ PD presenta subestructura óptima: Solución óptima para el problema esta constituido de soluciones óptimas de los subproblemas.
- Ventaja: Una vez solucionado un subproblema, la respuesta se guarda en una tabla y nunca mas se debe recalcular.
- Implementación: Puedes ser iterativa o recursiva, usando tablas de "memoización" para guardar los resultados parciales y así evitar recalcular.

- Número de Fibonacci:

- Número de Fibonacci:

- ▶ Algoritmo recursivo: Complejidad exponencial $O(2^n)$

```
1: function FIBONACCIRECURSIVO(n)
2:   if n = 0 then
3:     Retornar 0
4:   else if n = 1 then
5:     Retornar 1
6:   else
7:     FibonacciRecursivo(n - 1) + FibonacciRecursivo(n - 2)
8:   end if
9: end function
```

- **Número de Fibonacci:**

- ▶ **Algoritmo recursivo:** Complejidad exponencial $O(2^n)$

```
1: function FIBONACCIRECURSIVO(n)
2:   if n = 0 then
3:     Retornar 0
4:   else if n = 1 then
5:     Retornar 1
6:   else
7:     FibonacciRecursivo(n - 1) + FibonacciRecursivo(n - 2)
8:   end if
9: end function
```

- ▶ **Ejemplo:**

n = 12

Muchos cálculos.

Árbol de llamadas recursivas para Fibonacci($n=6$)

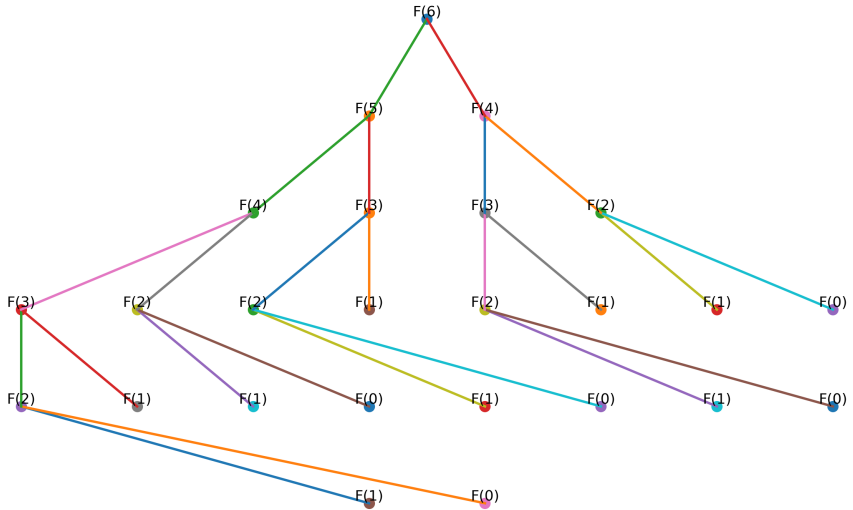


Figura 2: $F(6) = 8$.

Árbol de llamadas recursivas para Fibonacci(n=8)

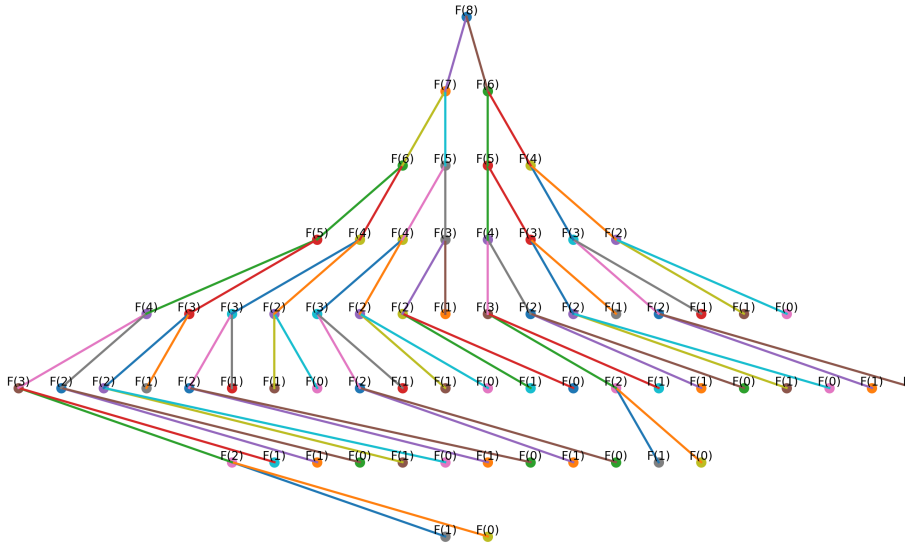


Figura 3: $F(6) = 21$.

- Número de Fibonacci: (cont.)

- **Número de Fibonacci:** (cont.)

- ▶ **Programación dinámica:** Complejidad lineal $O(n)$

```
1: procedure FIBONACCIDP( $n$ )
2:   Crear un arreglo fib de tamaño  $n + 1$ 
3:   fib[0]:= 0
4:   if  $n \geq 1$  then
5:     fib[1]:= 1
6:   end if
7:   for  $i := 2 : n$  do
8:     fib[i]:= fib[i-1] + fib[i-2]
9:     Mostrar Iter  $i$ :  $F(i) = F(i-1) + F(i-2) = \text{fib}[i-1] + \text{fib}[i-2] = \text{fib}[i]$ 
10:  end for
11:  Mostrar Sucesión de Fibonacci: fib[0] hasta fib[n]
12:  Mostrar Resultado final:  $F(n) = \text{fib}[n]$ 
13:  Return fib[n]
14: end procedure
```

- Número de Fibonacci: (cont.)

● Número de Fibonacci: (cont.)

► Ejemplo:

n = 12

Tabla:

$$\text{Iter 2: } F(2) = F(1) + F(0) = 1 + 0 = 1$$

$$\text{Iter 3: } F(3) = F(2) + F(1) = 1 + 1 = 2$$

$$\text{Iter 4: } F(4) = F(3) + F(2) = 2 + 1 = 3$$

$$\text{Iter 5: } F(5) = F(4) + F(3) = 3 + 2 = 5$$

$$\text{Iter 6: } F(6) = F(5) + F(4) = 5 + 3 = 8$$

$$\text{Iter 7: } F(7) = F(6) + F(5) = 8 + 5 = 13$$

$$\text{Iter 8: } F(8) = F(7) + F(6) = 13 + 8 = 21$$

$$\text{Iter 9: } F(9) = F(8) + F(7) = 21 + 13 = 34$$

$$\text{Iter 10: } F(10) = F(9) + F(8) = 34 + 21 = 55$$

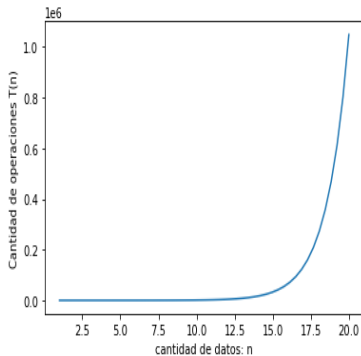
$$\text{Iter 11: } F(11) = F(10) + F(9) = 55 + 34 = 89$$

$$\text{Iter 12: } F(12) = F(11) + F(10) = 89 + 55 = 144$$

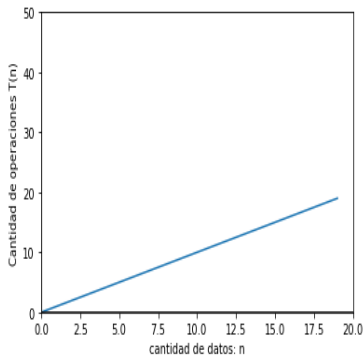
Sucesión de Fibonacci:

[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144]

Resultado final: $F(12) = 144$



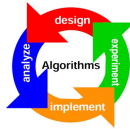
(a) $T(n) = 2^n$



(b) $T(n) = n$

Figura 4: Funciones de complejidad para Num. de Fibonacci.

Programación dinámica



- 1 Programación dinámica
- 2 Aplicaciones
- 3 Problema de la mochila binaria

- Optimización de secuencias:

¹Longest Common Subsequence

- Optimización de secuencias:
 - ▶ Edición de cadenas (string edit distance): Usado para corrección ortográfica y procesamiento de lenguaje natural.

¹Longest Common Subsequence

- Optimización de secuencias:

- ▶ Edición de cadenas (string edit distance): Usado para corrección ortográfica y procesamiento de lenguaje natural.
- ▶ Alineamiento de secuencias de ADN/ARN: En bioinformática.

¹Longest Common Subsequence

- Optimización de secuencias:

- ▶ Edición de cadenas (string edit distance): Usado para corrección ortográfica y procesamiento de lenguaje natural.
- ▶ Alineamiento de secuencias de ADN/ARN: En bioinformática.
- ▶ Subsecuencia común más larga:¹ Se usa en comparación de documentos o cadenas de texto.

¹Longest Common Subsequence

- Optimización de secuencias:

- ▶ Edición de cadenas (string edit distance): Usado para corrección ortográfica y procesamiento de lenguaje natural.
- ▶ Alineamiento de secuencias de ADN/ARN: En bioinformática.
- ▶ Subsecuencia común más larga:¹ Se usa en comparación de documentos o cadenas de texto.
- ▶ Programación predictiva del plegamiento de proteínas: Usado en biología computacional.

¹Longest Common Subsequence

● Optimización de secuencias:

- ▶ Edición de cadenas (string edit distance): Usado para corrección ortográfica y procesamiento de lenguaje natural.
- ▶ Alineamiento de secuencias de ADN/ARN: En bioinformática.
- ▶ Subsecuencia común más larga:¹ Se usa en comparación de documentos o cadenas de texto.
- ▶ Programación predictiva del plegamiento de proteínas: Usado en biología computacional.
- ▶ Corte de Barras (Rod Cutting Problem): Maximizar el beneficio al cortar una barra en piezas de longitud específica con valores dados.

¹Longest Common Subsequence

- **Optimización de secuencias:**

- ▶ **Edición de cadenas (string edit distance):** Usado para corrección ortográfica y procesamiento de lenguaje natural.
- ▶ **Alineamiento de secuencias de ADN/ARN:** En bioinformática.
- ▶ **Subsecuencia común más larga:**¹ Se usa en comparación de documentos o cadenas de texto.
- ▶ **Programación predictiva del plegamiento de proteínas:** Usado en biología computacional.
- ▶ **Corte de Barras (Rod Cutting Problem):** Maximizar el beneficio al cortar una barra en piezas de longitud específica con valores dados.
- ▶ **Emparejamiento Óptimo (Optimal Matching):** Hallar el mejor emparejamiento entre dos conjuntos. Ejemplo: Hospitales y médicos.

¹Longest Common Subsequence

- Asignación y Empaquetamiento:

- **Asignación y Empaquetamiento:**

- ▶ **Problema de la mochila (Knapsack Problem):** Se usa en finanzas, logística, y selección de proyectos.

- **Asignación y Empaquetamiento:**

- ▶ **Problema de la mochila (Knapsack Problem):** Se usa en finanzas, logística, y selección de proyectos.
- ▶ **Empaquetado de Contenedores (Bin Packing):** Colocar ítems de diferentes tamaños en un número limitado de contenedores de manera que se minimice su número total a ser utilizados.

● Asignación y Empaquetamiento:

- ▶ **Problema de la mochila (Knapsack Problem):** Se usa en finanzas, logística, y selección de proyectos.
- ▶ **Empaquetado de Contenedores (Bin Packing):** Colocar ítems de diferentes tamaños en un número limitado de contenedores de manera que se minimice su número total a ser utilizados.
- ▶ **Bin Packing y Cutting Stock:** Optimización de uso de materiales en la industria.

● Asignación y Empaquetamiento:

- ▶ **Problema de la mochila (Knapsack Problem):** Se usa en finanzas, logística, y selección de proyectos.
- ▶ **Empaquetado de Contenedores (Bin Packing):** Colocar ítems de diferentes tamaños en un número limitado de contenedores de manera que se minimice su número total a ser utilizados.
- ▶ **Bin Packing y Cutting Stock:** Optimización de uso de materiales en la industria.
- ▶ **Asignación óptima de recursos (Budget Allocation):** Maximizar beneficios bajo restricciones.

- Optimización de Rutas:

- Optimización de Rutas:

- ▶ Ruta corta con restricciones: Es una versión más compleja que Dijkstra y se, usa en robótica o juegos.

- Optimización de Rutas:

- ▶ Ruta corta con restricciones: Es una versión más compleja que Dijkstra y se, usa en robótica o juegos.
- ▶ Problema del viajante (TSP) con memoización: Generar ruta de menor costo.

● Optimización de Rutas:

- ▶ Ruta corta con restricciones: Es una versión más compleja que Dijkstra y se, usa en robótica o juegos.
- ▶ Problema del viajante (TSP) con memoización: Generar ruta de menor costo.
- ▶ Ruteo de Vehículos (Vehicle Routing): Planificar rutas óptimas para vehículos que deben visitar múltiples clientes, empezando y terminando el proceso en un centro llamado depósito.

● Optimización de Rutas:

- ▶ **Ruta corta con restricciones:** Es una versión más compleja que Dijkstra y se, usa en robótica o juegos.
- ▶ **Problema del viajante (TSP) con memoización:** Generar ruta de menor costo.
- ▶ **Ruteo de Vehículos (Vehicle Routing):** Planificar rutas óptimas para vehículos que deben visitar múltiples clientes, empezando y terminando el proceso en un centro llamado depósito.
- ▶ **Problema de recolección monedas:**² Rutas máximas en un tablero. Este problema se usa en videojuegos y robótica.

²Coin-collection

- Planificación:

- **Planificación:**

- ▶ **Planificación de Producción:** Decidir cuánto producir en cada período para atender la demanda y la minimización de sus costos.

- **Planificación:**

- ▶ **Planificación de Producción:** Decidir cuánto producir en cada período para atender la demanda y la minimización de sus costos.
- ▶ **Planeamiento de Proyectos (Project Scheduling):** Determinar el calendario óptimo de actividades con restricciones de precedencia.

● Planificación:

- ▶ **Planificación de Producción:** Decidir cuánto producir en cada período para atender la demanda y la minimización de sus costos.
- ▶ **Planeamiento de Proyectos (Project Scheduling):** Determinar el calendario óptimo de actividades con restricciones de precedencia.
- ▶ **Planeamiento de Personal (Staff Scheduling):** Asignar turnos al personal para cumplir con requerimientos y minimizar costos de sobretiempo.

● Planificación:

- ▶ **Planificación de Producción:** Decidir cuánto producir en cada período para atender la demanda y la minimización de sus costos.
- ▶ **Planeamiento de Proyectos (Project Scheduling):** Determinar el calendario óptimo de actividades con restricciones de precedencia.
- ▶ **Planeamiento de Personal (Staff Scheduling):** Asignar turnos al personal para cumplir con requerimientos y minimizar costos de sobretiempo.
- ▶ **Planeamiento de Cultivos o Recursos Naturales:** Maximizar la producción agrícola, forestal o pesquera sujeta a restricciones de tiempo y espacio.

● Planificación:

- ▶ **Planificación de Producción:** Decidir cuánto producir en cada período para atender la demanda y la minimización de sus costos.
- ▶ **Planeamiento de Proyectos (Project Scheduling):** Determinar el calendario óptimo de actividades con restricciones de precedencia.
- ▶ **Planeamiento de Personal (Staff Scheduling):** Asignar turnos al personal para cumplir con requerimientos y minimizar costos de sobretiempo.
- ▶ **Planeamiento de Cultivos o Recursos Naturales:** Maximizar la producción agrícola, forestal o pesquera sujeta a restricciones de tiempo y espacio.
- ▶ **Planeamiento de Energía (Unit Commitment Problem):** Es un problema de optimización en la planificación energética. Objetivo: Decidir qué unidades generadoras de energía deben estar operativas en un período determinado para atender la demanda de electricidad de manera eficiente y económica.

- Inteligencia Artificial y Aprendizaje Automático:

- **Inteligencia Artificial y Aprendizaje Automático:**
 - ▶ **Decisiones secuenciales en ambientes estocásticos:** En Reinforcement Learning.

- **Inteligencia Artificial y Aprendizaje Automático:**

- ▶ **Decisiones secuenciales en ambientes estocásticos:** En Reinforcement Learning.
- ▶ **Segmentación óptima:** Usado en modelos ocultos de Markov para reconocimiento de voz y lenguaje.

- **Inteligencia Artificial y Aprendizaje Automático:**

- ▶ **Decisiones secuenciales en ambientes estocásticos:** En Reinforcement Learning.
- ▶ **Segmentación óptima:** Usado en modelos ocultos de Markov para reconocimiento de voz y lenguaje.
- ▶ **Planificación Óptima en Agentes Inteligentes:** Determinar la secuencia óptima de acciones que debe tomar un agente en un entorno con incertidumbre.

● Inteligencia Artificial y Aprendizaje Automático:

- ▶ **Decisiones secuenciales en ambientes estocásticos:** En Reinforcement Learning.
- ▶ **Segmentación óptima:** Usado en modelos ocultos de Markov para reconocimiento de voz y lenguaje.
- ▶ **Planificación Óptima en Agentes Inteligentes:** Determinar la secuencia óptima de acciones que debe tomar un agente en un entorno con incertidumbre.
- ▶ **Visión por Computadora:** En tareas de segmentación o análisis de secuencias de imágenes, se aplica PD para calcular alineamientos o trayectorias óptimas. Por ejemplo: Seguimiento de objetos (object tracking) mediante detección de camino óptimo en un espacio de estados.

- Procesamiento de Datos y Visualización

● Procesamiento de Datos y Visualización

- ▶ **Compresión de datos:** Técnicas usadas para reducir el tamaño de los datos, eliminando redundancias y aprovechando soluciones óptimas previas. Esto ayuda a almacenar o transmitir información de forma eficiente.

● Procesamiento de Datos y Visualización

- ▶ **Compresión de datos:** Técnicas usadas para reducir el tamaño de los datos, eliminando redundancias y aprovechando soluciones óptimas previas. Esto ayuda a almacenar o transmitir información de forma eficiente.
- ▶ **Reconocimiento de patrones:** Técnicas usadas para identificar y clasificar patrones de datos. Es importante en IA, aprendizaje automático y procesamiento de datos. En general implica el uso de algoritmos para analizar datos y hallar estructuras que puedan ser útiles clasificar, predecir o tomar decisiones.

● Procesamiento de Datos y Visualización

- ▶ **Compresión de datos:** Técnicas usadas para reducir el tamaño de los datos, eliminando redundancias y aprovechando soluciones óptimas previas. Esto ayuda a almacenar o transmitir información de forma eficiente.
- ▶ **Reconocimiento de patrones:** Técnicas usadas para identificar y clasificar patrones de datos. Es importante en IA, aprendizaje automático y procesamiento de datos. En general implica el uso de algoritmos para analizar datos y hallar estructuras que puedan ser útiles clasificar, predecir o tomar decisiones.
- ▶ **Interpolación de curvas óptimas (como B-splines):** Diseño gráfico y CAD.

● Procesamiento de Datos y Visualización

- ▶ **Compresión de datos:** Técnicas usadas para reducir el tamaño de los datos, eliminando redundancias y aprovechando soluciones óptimas previas. Esto ayuda a almacenar o transmitir información de forma eficiente.
- ▶ **Reconocimiento de patrones:** Técnicas usadas para identificar y clasificar patrones de datos. Es importante en IA, aprendizaje automático y procesamiento de datos. En general implica el uso de algoritmos para analizar datos y hallar estructuras que puedan ser útiles clasificar, predecir o tomar decisiones.
- ▶ **Interpolación de curvas óptimas (como B-splines):** Diseño gráfico y CAD.

● Optimización de Actividades

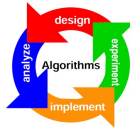
● Procesamiento de Datos y Visualización

- ▶ **Compresión de datos:** Técnicas usadas para reducir el tamaño de los datos, eliminando redundancias y aprovechando soluciones óptimas previas. Esto ayuda a almacenar o transmitir información de forma eficiente.
- ▶ **Reconocimiento de patrones:** Técnicas usadas para identificar y clasificar patrones de datos. Es importante en IA, aprendizaje automático y procesamiento de datos. En general implica el uso de algoritmos para analizar datos y hallar estructuras que puedan ser útiles clasificar, predecir o tomar decisiones.
- ▶ **Interpolación de curvas óptimas (como B-splines):** Diseño gráfico y CAD.

● Optimización de Actividades

- ▶ **Selección de Actividades (Activity Selection):** Dados un conjunto de actividades; horario de inicio y fin, se desea seleccionar el máximo número de actividades con la condición que no se superpongan.

Programación dinámica



- 1 Programación dinámica
- 2 Aplicaciones
- 3 Problema de la mochila binaria

Problema de la mochila binaria

- Conjunto de n itens de peso: $s_1, s_2, \dots, s_n$:

¿Existirá un subconjunto de itens con un peso total de exactamente S ?

Problema de la mochila binaria

- **Conjunto de n itens de peso:** $s_1, s_2, \dots, s_n$:

¿Existirá un subconjunto de itens con un peso total de exactamente S ?

- **Guardar la mochila en una tabla:** $t[i, j]$ es un conjunto verdadero *si y solamente si*

(a) $t[i - 1, j]$ es verdadero.

(b) $t[i - 1, j - s_i]$ tiene sentido y es verdadero.

- Algoritmicamente:

- 1: $t[i-1, j]$
- 2: **if** $j - s_i \geq 0$ **then**
- 3: $t[i, j] := t[i, j] \text{ o } t[i-1, j - s_i]$
- 4: **end if**

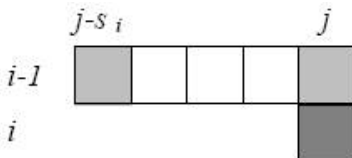


Figura 5: Almacenando items.

- **Llenado de la tabla:** Sucesión de valores verdaderos (t) y falsos (f).

$[i, j]$	0	1									S
0	t	f	f	f	f	f	f	f	f	f	f
1											
2											
3											
4											
5											
$\vdots$											
n											

- Seudocódigo:

$t[i, j]$ es verdadero, sí existe un subconjunto de los primeros i items que tiene una longitud exacta j .

1: Knapsack($(s_1, s_2, \dots, s_n), S$)

2: $t[0, 0] := t$

▷ Verdadero o true o 1

3: **for** $j = 1 : S$ **do**

4: $t[0, j] := f$

▷ Falso o false o 0

5: **end for**

6: **for** $i = 1 : n$ **do**

7: **for** $j = 0 : S$ **do**

8: $t[i, j] := t[i - 1, j]$

9: **if** $j - s_i \geq 0$ **then**

10: $t[i, j] := t[i, j]$ o $t[i - 1, j - s_i]$

11: **end if**

12: **end for**

13: **end for**

14: Retornar $t[n, S]$

- Ejemplo:

- Ejemplo:

- ▶ Mochila: Con itens y sus respectivos pesos:

- Ejemplo:

- ▶ Mochila: Con itens y sus respectivos pesos:

- ★ $s_1 = 1$; $s_2 = 2$; $s_3 = 2$; $s_4 = 4$; $s_5 = 5$; $s_6 = 2$; $s_7 = 4$.

- Ejemplo:

- ▶ Mochila: Con itens y sus respectivos pesos:

- ★ $s_1 = 1; s_2 = 2; s_3 = 2; s_4 = 4; s_5 = 5; s_6 = 2; s_7 = 4.$

- ★ Peso total: $S = 15.$

● Ejemplo:

- **Mochila:** Con itens y sus respectivos pesos:

★ $s_1 = 1; s_2 = 2; s_3 = 2; s_4 = 4; s_5 = 5; s_6 = 2; s_7 = 4.$

★ Peso total: $S = 15.$

- **Para llenar la tabla usar:** $t[i, j] := t[i - 1, j] \text{ o } t[i - 1, j - s_i]$

$$t[3, 3] := t[2, 3] \text{ o } t[2, 3 - s_3] = t[2, 3] \text{ o } t[2, 3 - 2] = t[2, 3] \text{ o } t[2, 1]$$

$[i, j]$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	15
0	t	f	f	f	f	f	f	f	f	f	f	f	f	f	f
1	t	t	f	f	f	f	f	f	f	f	f	f	f	f	f
2	t	t	t	t	f	f	f	f	f	f	f	f	f	f	f
3	t	t	t	t											
4															
5															
6															
7															

- Ejemplo (cont.):

● Ejemplo (cont.):

- Para llenar la tabla usar: $t[i, j] := t[i - 1, j] \text{ o } t[i - 1, j - s_i]$ (cont.)

$$t[3, 4] := t[2, 4] \text{ o } t[2, 4 - s_3] = t[2, 4] \text{ o } t[2, 2]$$

$[i, j]$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	15
0	t	f	f	f	f	f	f	f	f	f	f	f	f	f	f
1	t	t	f	f	f	f	f	f	f	f	f	f	f	f	f
2	t	t	t	t	f	f	f	f	f	f	f	f	f	f	f
3	t	t	t	t	t										
4															
5															
6															
7															

- Ejemplo (cont.):

- Ejemplo (cont.):

- ▶ Tabla llena

$[i,j]$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	15
0	t	f	f	f	f	f	f	f	f	f	f	f	f	f	f
1	t	t	f	f	f	f	f	f	f	f	f	f	f	f	f
2	t	t	t	t	f	f	f	f	f	f	f	f	f	f	f
3	t	t	t	t	t	t	f	f	f	f	f	f	f	f	f
4	t	t	t	t	t	t	t	t	t	t	f	f	f	f	f
5	t	t	t	t	t	t	t	t	t	t	t	t	t	t	f
6	t	t	t	t	t	t	t	t	t	t	t	t	t	t	t
7	t	t	t	t	t	t	t	t	t	t	t	t	t	t	t

- Ejemplo (cont.):

- Ejemplo (cont.):

- ▶ Tabla con itens i y sus pesos j cuyos valores son verdaderos:

$[i, j]$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	15
0	t														
1	t	t													
2	t	t	t	t											
3	t	t	t	t	t	t									
4	t	t	t	t	t	t	t	t	t	t					
5	t	t	t	t	t	t	t	t	t	t	t	t	t	t	
6	t	t	t	t	t	t	t	t	t	t	t	t	t	t	t
7	t	t	t	t	t	t	t	t	t	t	t	t	t	t	t

- Ejemplo (cont.):

- Ejemplo (cont.):

- ▶ Tabla con los primeros ítems i y sus pesos j hasta llenar la mochila:

$[i, j]$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	15
0															
1		s_1													
2			s_2												
3			s_3												
4					s_4										
5															
6			s_6												
7					s_7										

- Ejemplo (cont.):

- ▶ Tabla con los primeros ítems i y sus pesos j hasta llenar la mochila:

$[i, j]$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	15
0															
1		s_1													
2			s_2												
3			s_3												
4					s_4										
5															
6			s_6												
7					s_7										

- ▶ Recordar: Con ítems y sus respectivos pesos:

- **Ejemplo (cont.):**

- ▶ Tabla con los primeros ítems i y sus pesos j hasta llenar la mochila:

$[i, j]$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	15
0															
1		s_1													
2			s_2												
3			s_3												
4					s_4										
5															
6			s_6												
7					s_7										

- ▶ **Recordar:** Con ítems y sus respectivos pesos:

★ $s_1 = 1; s_2 = 2; s_3 = 2; s_4 = 4; s_5 = 5; s_6 = 2; s_7 = 4.$

- Ejemplo (cont.):

- ▶ Tabla con los primeros ítems i y sus pesos j hasta llenar la mochila:

$[i, j]$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	15
0															
1		s_1													
2			s_2												
3			s_3												
4					s_4										
5															
6			s_6												
7					s_7										

- ▶ **Recordar:** Con ítems y sus respectivos pesos:

★ $s_1 = 1; s_2 = 2; s_3 = 2; s_4 = 4; s_5 = 5; s_6 = 2; s_7 = 4.$

★ Peso total: $S = 15.$

- Ejemplo:

- Ejemplo:

- ▶ Mochila: Con itens y sus respectivos pesos:

- Ejemplo:

- ▶ Mochila: Con itens y sus respectivos pesos:

- ★ $s_1 = 5$; $s_2 = 4$; $s_3 = 3$; $s_4 = 6$.

- Ejemplo:

- ▶ Mochila: Con itens y sus respectivos pesos:

- ★ $s_1 = 5; s_2 = 4; s_3 = 3; s_4 = 6.$

- ★ Peso total: $S = 7.$

- Ejemplo:

- ▶ Mochila: Con itens y sus respectivos pesos:

- ★ $s_1 = 5; s_2 = 4; s_3 = 3; s_4 = 6$.

- ★ Peso total: $S = 7$.

- ▶ Para llenar la tabla usar: $t[i, j] := t[i - 1, j]$ o $t[i - 1, j - s_i]$

$[i, j]$	0	1	2	3	4	5	6	7
0	t	f	f	f	f	f	f	f
1	t	f	f	f	f	t	f	f
2	t	f	f	f	t	t	f	f
3	t	f	f	t	t	t	f	t
4	t	f	f	t	t	t	t	t

- Ejemplo (cont.):

- Ejemplo (cont.):

- ▶ Tabla con itens i y sus pesos j

$[i, j]$	0	1	2	3	4	5	6	7
0	t							
1	t					t		
2	t				t	t		
3	t			t	t	t		t
4	t			t	t	t	t	t

- Ejemplo (cont.):

- Ejemplo (cont.):

- ▶ Tabla con los primeros itens i y sus pesos j hasta llenar la mochila:

$[i, j]$	0	1	2	3	4	5	6	7
0								
1								
2					s_2			
3				s_3				
4								